

# String in Python

- A sequence is an ordered collection of items, indexed by positive integers.
- It is a combination of mutable (a mutable variable is one, whose value may change) and immutable (an immutable variable is one, whose value may not change) data type.
- There are 3 type of sequence available in python; they are Strings, List and Tuple.

- A string is a sequence of one or more characters (letters, numbers, symbols) that can be either a constant or a variable.
- String are the form of data used in programming for storing and manipulating text such as words, name, and sentence.
- String is immutable data types i.e. they are unchanging .
- You can write them in python using single quotes, double quotes or triple quotes.
- Example:

`'infomax'`  
`'''infomax'''`

`"infomax"`  
`"""infomax"""`

- In python each element (character) of string access by its index (position), which is unique for all element.
- Positive index always start with 0 so the last index is size-1.
- Negative start with size\*-1 so the last index is -1
- For example if : `str1="INFOMAX"` then representation is

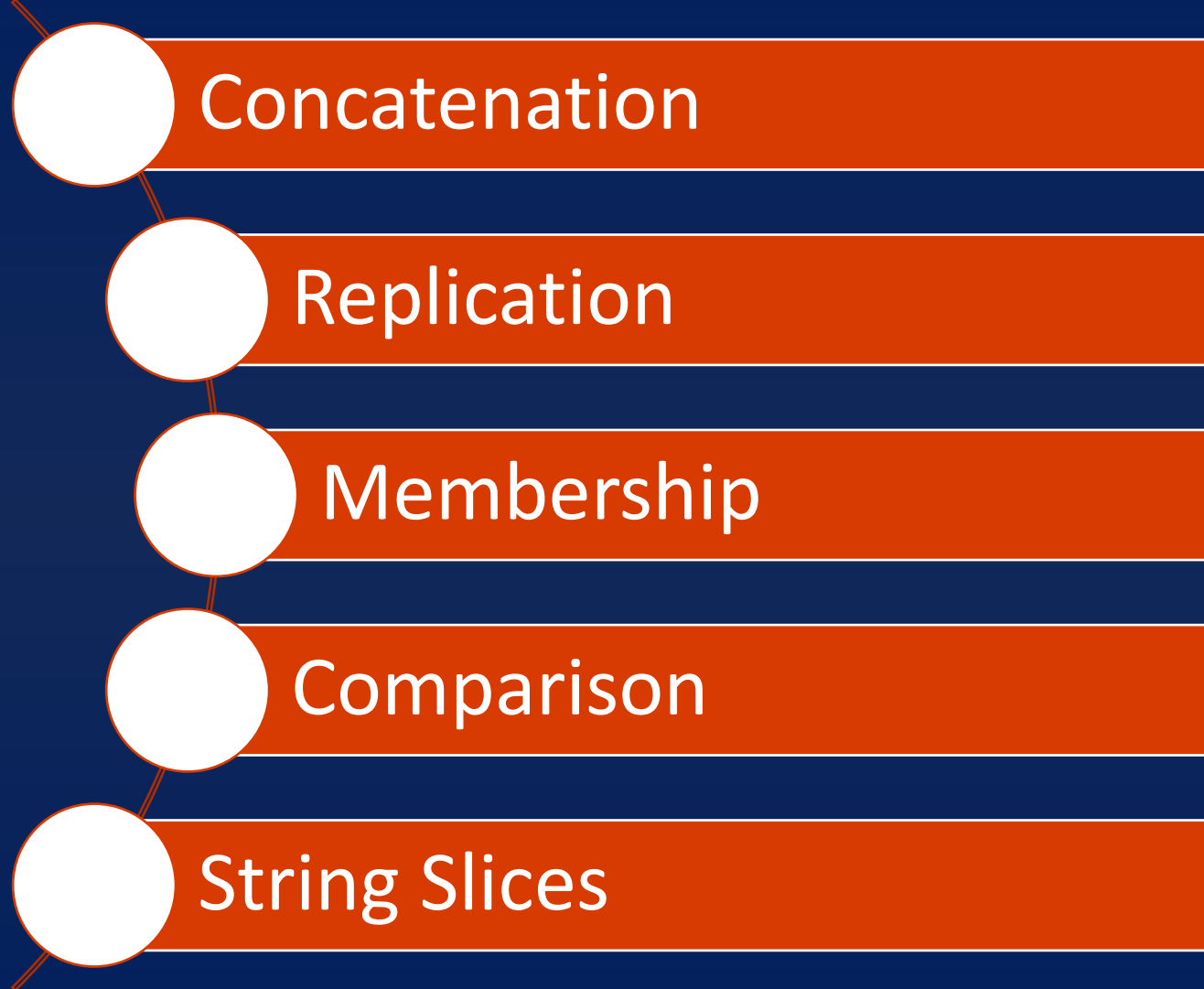
| str1           | I  | N  | F  | O  | M  | A  | X  |
|----------------|----|----|----|----|----|----|----|
| Positive Index | 0  | 1  | 2  | 3  | 4  | 5  | 6  |
| Negative Index | -7 | -6 | -5 | -4 | -3 | -2 | -1 |

- Traversing a string means accessing all the elements of the string one after the another by using the subscript or index.

```
str1="infomax"  
for a in str1:  
    print(a)
```

```
str1="infomax"  
for a in range(0,len(str1)):  
    print(str1[a])
```

```
str1="infomax"  
for a in range(len(str1)*-1,0):  
    print(str1[a])
```



# Concatenation Operator



➤ The + operator creates a new string by joining the two operand string  
e.g. 'info' +'max' will result into : infomax

|               |                  |
|---------------|------------------|
| ➤ Expression: | will result into |
| ➤ '1'+'1'     | '11'             |
| ➤ "a"+"0"     | 'a0'             |
| ➤ '123'+'abc' | '123abc'         |

- The \* operator when used with numbers (i.e. when both operands are numbers), it performs multiplication and returns the product of the two numbers.
- To use a \* operator with string, you need two types of operands—a string and a number, i.e. as:

*number\*string* or *string\*number*

## Expression

"abc"\*2

5\* "@"

"\$"\*4

"2"\*3

## will result into

"abcabc"

"@@@@@"

"\$\$\$\$"

"222"



- There are two membership operators for string (in fact for all sequence type).
- These are **in** and **not in**
- **in** : Return True if a character or a substring exist in the given string; False otherwise
- **not in** : Return True if a character or a substring not exist in the given string; False otherwise

- Python's standard comparison operators i.e. all relational operators (<,<=,>,>=,==,!=) apply to string also.
- The comparison of character is performed based on ASCII value.

| ➤ Expression    | will gives you |
|-----------------|----------------|
| ➤ "abc"=="abc"  | True           |
| ➤ "Abc"=="abc"  | False          |
| ➤ "ABC"!="abc"  | True           |
| ➤ 'ABC'>'AB'    | True           |
| ➤ 'a'<'A'       | False          |
| ➤ 'abcd'>'abcD' | True           |

➤ Slicing refers to a part of the string, where strings are sliced using a range of indices (index).

➤ Syntax:

`string_name[n:m]` where n and m are integer and legal index.

|      |    |    |    |    |    |    |    |
|------|----|----|----|----|----|----|----|
|      | 0  | 1  | 2  | 3  | 4  | 5  | 6  |
| word | i  | n  | f  | o  | m  | a  | x  |
|      | -7 | -6 | -5 | -4 | -3 | -2 | -1 |

`word[0:3]`      inf

`word[2:5]`      fom

`word[-7:-3]`    info

`word[:5]`      infom

`word[3:]`      omax

`word[::-1]`    xamofni

`word[]`        infomax

- **len()** method returns the length of a string:

```
a = "Hello, World!"  
print(len(a)) # 13
```

- **lower()** method returns the string in lower case:

```
a = "Hello, World!"  
print(a.lower()) # hello, world!
```

- **upper()** method returns the string in upper case:

```
a = "Hello, World!"  
print(a.upper()) # HELLO, WORLD!
```

- **capitalize()** method returns a copy of the string with its first character capitalized.

```
print("i love my India". capitalize()) # I love my India
```

- **isalpha()** Return true if all character in the string are alphabetic and there is at least one character, false otherwise.

```
print("raju".isalpha()) # True
```

- **isdigit()** Return true if all character in the string are digits and there is at least one character, False otherwise.

```
➤ print("123".isdigit()) # True
```

➤ **isalnum()** Return True if the character in the string are alphanumeric (alphabets or number) and there is at least one character, False otherwise.

```
print("abc123".isalnum()) # True
```

➤ **isspace()** Return True if there are only whitespace characters in the string. There are must be at least one character, False otherwise.

```
print(" ".isspace()) # True
```

➤ **islower()** Return true if all cased characters in the string are lowercase.

```
print("infomax".islower()) # True
```

➤ **isupper()** Return true if all cased characters in the string are uppercase.

```
print("INFOMAX".isupper()) # True
```

➤ **Find()** it return the position of the given substring within the string; it return -1 if it is unsuccessful in finding the substring.

```
st="infomax computer academy"
```

```
print(st.find("max")) #4
```

```
print(st.find("o",7,15)) # 9
```

```
print(st.find("max",10,25)) # -1
```

➤ **replace()** method replaces a string with another string:

```
print("Hello, World!".replace("H", "J"))    # Jello, World!
```

➤ **lstrip()** : Returns the string after removing the space(s) on the left of the string.

```
print("  infomax".lstrip())    #infomax
```

➤ **rstrip()** : Returns the string after removing the space(s) on the right of the string.

```
➤ print("  infomax  ".rstrip())    # infomax
```



➤ **istitle()** : Returns True if the string is title cased. Otherwise returns False

```
print("The Green Revolution".istitle())    # True
```

➤ **join ()** : Returns a string in which the string elements have been joined by a separator.

```
str1=('jan', 'feb' , 'mar')  
print(str.join("&")) # 'jan&feb&mar'
```

➤ **swapcase()** : Returns the string with case changes.

```
print("Infomax".swapcase()) # iNFOMAX
```

➤ **strip()** method removes any whitespace from the beginning or the end:

```
a = " Hello, World! "
```

```
print(a.strip()) # returns "Hello, World!"
```

➤ **split()** method splits the string into substrings if it finds instances of the separator:

```
a = "Hello, World!"
```

```
print(a.split(",")) # returns ['Hello', ' World!']
```

➤ **partition(sep)** : The function partitions the strings at the first occurrence of separator, and returns the strings partition in three parts i.e. before the separator, the separator itself, and the part after the separator. If the separator is not found, returns the string itself, followed by two empty strings.

```
>>> str='The Green Revolution'  
>>> str.partition('Rev')  
( 'The Green ', 'Rev', 'olution' )  
>>> str.partition('pe')  
( 'The Green Revolution', '', '' )  
>>> str.partition('e')  
( 'Th', 'e', ' Green Revolution' )
```

- WAP to print the string in reverse.
- WAP to check input string is a palindrome or not.
- Write a program to print alternate characters in a string.